



Modern endpoint device management

Managing devices as code

Contents

- 3 Introduction
- 3 The rise of decentralized device fleets
- 3 The burden of traditional endpoint management
- 4 Transition to code-based infrastructure
- 6 The GitOps framework for devices
- 7 Comparison of management methodologies
- 8 Strategic benefits for the organization
- 9 Implement IaC and GitOps
- 9 The road ahead for device management
- 10 Detailed exploration of the GitOps ecosystem



Introduction

The management of enterprise IT infrastructure has undergone a radical transformation. Organizations no longer operate within the confines of a single office building. Instead, they support a decentralized workforce spread across homes, satellite offices, and mobile environments.¹ This shift means IT leaders must now maintain control over thousands of devices that require constant security updates, visibility, and compliance monitoring.¹ Traditional on-premises tools, designed for static and small environments, often fail to meet these modern demands.¹ This report examines the transition from manual, reactive processes to proactive, code-based strategies known as Infrastructure as Code (IaC) and GitOps.

The rise of decentralized device fleets

In the past, device management was a background task. IT teams focused on hardware procurement and occasional software patches. Today, endpoint management is a core operational priority.¹ The modern fleet includes a fragmented mix of operating systems, including Windows, macOS, and Linux, alongside mobile platforms like iOS and Android.² This variety creates a complex environment where standardizing policies across different ownership models, such as company-owned or Bring Your Own Device (BYOD) becomes a significant hurdle.³

The consequence of this complexity is a strained IT staff. Manual scaling of updates and security policies leads to delays and widens the attack surface.² When IT teams lack a unified platform to manage every device, security management suffers, and threats go undetected.³ This evolution of the workplace requires a move toward cloud-native management platforms that offer visibility and automation from a single dashboard.¹

The burden of traditional endpoint management

IT leaders face specific challenges when they rely on legacy management methods. These challenges impact not only the organization's security posture but also the overall operational budget and employee productivity.

Business challenges in a hybrid world

The move to remote and hybrid work means many devices never connect directly to a corporate network.³ This lack of direct connection creates visibility and control gaps.⁴ IT administrators often struggle to answer basic questions: which applications are installed, which operating systems require patches, and which endpoints have slipped out of view.⁴ Security risks evolve faster than traditional models can handle.³ Endpoints serve as the primary entry point for over 75 percent of security breaches.⁴ Manual tracking through spreadsheets or stale inventories is no longer effective.⁴ Ransomware, malware, and phishing attacks target these unmanaged or unpatched devices, potentially compromising the entire corporate network.²

Shadow IT and the visibility gap

Shadow IT occurs when employees use unauthorized applications or hardware.³ This behavior multiplies organizational risk because IT teams cannot monitor or patch what they cannot see.³ Without real-time inventory and live dashboards, security blind spots remain open for exploitation by sophisticated actors like advanced persistent threats (APTs).⁴ The administrative burden of managing numerous disparate solutions increases costs and complicates the security landscape.⁵

Compliance and regulatory pressure

Industry regulations like GDPR, HIPAA, and PCI-DSS demand strict data security and audit readiness.³ Proving compliance with manual tools is a cumbersome and time-consuming process.³ Failing to provide accurate reports can result in legal penalties and heavy fines.³ Organizations need a way to enforce encryption, access controls, and reporting automatically to meet these mandates without exhausting their personnel.³

Transition to code-based infrastructure

Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure through machine-readable configuration files rather than manual processes or physical hardware setup.⁷ While originally used for servers and cloud resources, IaC now applies to the management of laptop and desktop fleets.⁷ By treating device configurations like software code,

IT teams ensure that their environments remain repeatable, scalable, and consistent.⁷

Declarative versus imperative models

There are two primary styles of IaC: declarative and imperative. The choice between them defines the organizational approach to automation.

Imperative IaC defines the specific steps required to reach a desired state.⁷ This is similar to a traditional script that says, "First do A, then do B, then do C." This approach can be difficult to manage as the fleet grows because it requires precise knowledge of the starting state of every device.⁷

Declarative IaC defines the end state itself.⁷ The administrator writes code that describes what a device should look like - for example, which security settings must be enabled and which software should be installed.¹⁰ The IaC tool then identifies the difference between the current state and the desired state and applies only the necessary changes to align them.¹⁰ This model requires less programming knowledge and is more reusable across different environments.⁹

Core principles of Infrastructure as Code

The success of IaC relies on several key principles that improve the stability of IT systems. Idempotency is a critical concept in IaC.¹² An idempotent process ensures that if you run the same configuration multiple times, the result remains the same without creating unintended changes.¹² This reduces the risk of accidental modifications when an IT team pushes updates to a large fleet of devices.¹²

Another principle is the use of immutable infrastructure.⁹ In an immutable model, IT teams do not make changes to resources directly. Instead, they deploy new templates that replace old configurations.⁹ This eliminates "configuration drift," where a device's settings slowly change over time due to manual edits or user interference.⁸

IaC also supports "Policy as Code".⁷ This allows organizations to define security policies programmatically. For instance, a policy can automatically block a deployment if a device does not have disk encryption enabled.⁴ This "shifts left" the focus of security by preventing problems before they occur rather than detecting them afterward.⁹

The GitOps framework for devices

GitOps is an operational framework that uses Git as the single source of truth for infrastructure and device management.¹³ It combines the power of IaC with a collaborative workflow and automated synchronization.¹⁴ While IaC focuses on the "what," GitOps focuses on the "how" - specifically, how changes are reviewed, approved, and deployed.¹⁷

Git as the single source of truth

In a GitOps model, all device configurations live in a Git repository.⁸ This repository serves as the authoritative record of how every device in the fleet should look.¹³ Because Git tracks every change, IT leaders have a clear history of who modified a setting and when they did it.⁶ This transparency creates a natural audit trail and simplifies the troubleshooting process.⁶

The pull request as a quality gate

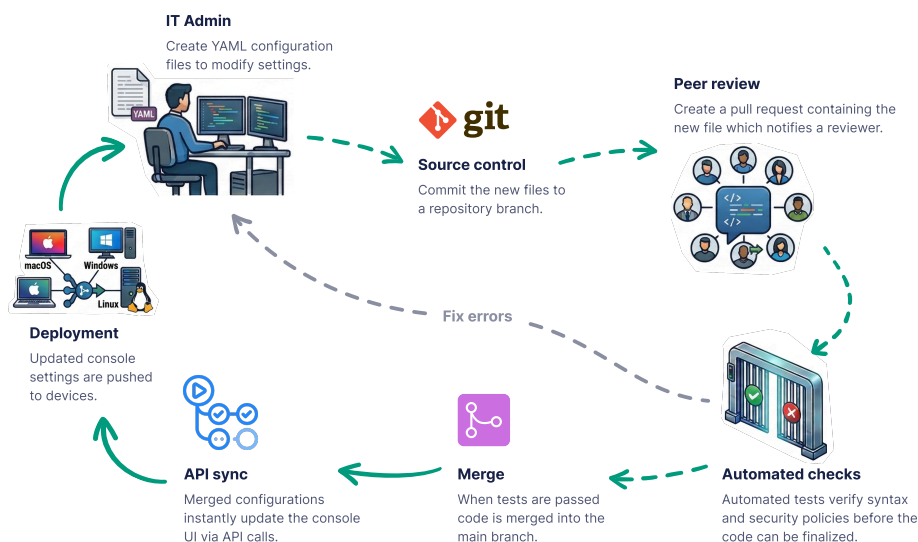
GitOps uses pull requests (PRs) or merge requests (MRs) as the primary mechanism for

change.¹⁴ When a team member wants to update a security policy or install a new application, they submit a PR with the proposed code changes.¹³ This creates a checkpoint where other team members can review the code, discuss its impact, and suggest improvements.⁸ Mandatory reviews act as a quality gate that prevents errors and ensures consistency across the organization.⁸

Automated reconciliation and self-healing

Once a change is approved and merged into the main branch of the repository, automated tools take over.¹³ These tools, such as controllers or operators, detect the change in Git and automatically update the devices to match the new state.¹³

One of the most powerful features of GitOps is continuous reconciliation.¹³ The system constantly compares the actual state of the device fleet against the desired state defined in Git.¹³ If a user manually changes a setting or a device drifts out of compliance, the GitOps controller automatically overwrites the manual change to restore the desired configuration.⁸



Comparison of management methodologies

The shift to code-based management represents a departure from traditional tools. The following table compares manual management, traditional Unified Endpoint Management (UEM), and GitOps-based management.

Git as the single source of truth

Feature	Manual management	Traditional UEM / MDM	IaC and GitOps
Primary interface	Device CLI or physical access	Web-based admin console	Git repository and code
Configuration state	Imperative (step-by-step)	Imperative or GUI-based	Declarative (desired state)
Source of truth	Disparate logs or spreadsheets	Vendor database	Version-controlled files
Change control	None or verbal approval	Admin action logs	Pull request and peer review
Scalability	Low (requires more staff)	Moderate (GUI bottlenecks)	High (automated pipelines)
Drift management	Manual discovery	Periodic reporting	Continuous self-healing
Audit trail	Fragmented or missing	History of admin clicks	Immutable Git commit history
Rollback process	Manual redo / undo	Backup and restore	Revert a Git commit
Security posture	Reactive and inconsistent	Policy-enforced (GUI)	Policy as Code (Automated)

Strategic benefits for the organization

Adopting IaC and GitOps provides IT leaders with significant advantages in terms of security, efficiency, and reliability.

Security and compliance at scale

By limiting direct access to systems and enforcing changes through a GitOps pipeline, organizations reduce their attack surface.⁸ Administrators no longer need to provide access credentials to every component of the infrastructure because automation handles the deployment.²⁰ Furthermore, secrets management tools can integrate directly with the pipeline to ensure sensitive information like passwords or API keys are handled securely.⁸

For compliance, GitOps transforms a cumbersome process into an automated one.⁶ Every update is tracked with a Git commit, providing a detailed record for auditors.¹³ This documentation-as-code approach ensures that system documentation stays current with the actual state of the devices.⁸

Operational efficiency and cost control

Automation eliminates the error-prone manual steps that often lead to downtime.⁸ By using code, IT teams can manage thousands of devices without a proportional increase in headcount.⁴ This efficiency allows professionals to focus on strategic initiatives instead of reactive troubleshooting.¹

Modernizing legacy systems through these methods can boost productivity by up to 33% and reduce maintenance costs by up to 25% in the first year.²¹ By consolidating multiple point solutions into a unified, code-based workflow, organizations avoid the high costs of fragmented tooling.⁴

Reliability and disaster recovery

GitOps significantly improves system stability.⁶ Because the entire environment is defined in code, recovering from a failure is much faster.⁹ If a change causes an unexpected issue, the team can instantly roll back to a known good state by reverting the last commit in Git.⁶ This reduces the mean time to recovery (MTTR) from hours to minutes.⁶

Implement IaC and GitOps

Transitioning to code-based management requires careful planning and a shift in team culture.

Device group planning and naming conventions

Before writing code, IT leaders must define their device groups and fleets. This involves segmenting devices by function, department, ownership model, or policy requirements — for example, separating engineering laptops from finance workstations, or corporate-owned devices from BYOD endpoints. Consistency is vital; establishing clear naming conventions for all device groups ensures that configurations and policies are applied to exactly the right set of machines, and avoids ambiguity when multiple teams manage overlapping segments of the same fleet.

Teams should also embrace the "Don't Repeat Yourself" (DRY) principle.¹² This means using modular designs and templates to provision resources.¹² By passing environment-specific variables into a single module, teams avoid duplicating code and reduce the chance of errors.¹²

The human element and team culture

GitOps requires a commitment to new processes.¹⁶ Teams must suppress the temptation to make manual, "cowboy engineering" changes to production devices.¹⁶ While the pull request process may seem slower at first, it fosters a collaborative culture where knowledge is shared and every change is reviewed for quality.⁸

Because Git is already the standard for software development, many IT professionals find the transition to GitOps relatively easy.⁶ It leverages existing skills and tools, making the learning curve less steep than adopting entirely new, proprietary platforms.²⁰

The road ahead for device management

As digital infrastructure continues to expand, the reliance on manual processes will become unsustainable.¹ Cloud-native platforms are becoming the foundation of modern IT operations, emphasizing simplicity, visibility, and scalability.¹

Emerging technologies, such as AI-powered automation, will likely further accelerate the ability to generate infrastructure code and monitor for threats in real time.²³

IT leaders who adopt Infrastructure as Code and GitOps position their organizations to be more resilient and secure. By treating management as a software engineering discipline, they gain the control needed to support a modern, distributed workforce. The move from manual clicks to version-controlled code ensures that the IT department remains an enabler of business growth rather than a bottleneck.

Detailed exploration of the GitOps ecosystem

The transition from traditional systems to GitOps is not merely a change in tooling; it is a fundamental shift in how IT operations are conceived and executed. To understand the depth of this shift, one must analyze the secondary and tertiary effects of moving to a code-backed management model.

In a traditional environment, the knowledge of how a system is configured often resides in the minds of a few senior administrators or is buried in poorly maintained documentation. When an administrator leaves the organization, that institutional knowledge often goes with them. GitOps solves this problem by ensuring that the "how" and "why" of every configuration are stored in the code and the Git commit messages.⁶ This "documentation as code" ensures that new team members can onboard quickly by reading the repository instead of relying on tribal knowledge.⁸

Furthermore, the "Pull-based" model of GitOps offers a distinct security advantage over traditional "Push-based" CI/CD systems. In a push model, the central automation server must have administrative credentials for every device it manages to "push" changes out. If the automation server is compromised, the entire fleet is at risk. In a pull-based GitOps model, a small agent or "operator" sits on the device or within the cluster and "pulls" configurations from the repository.⁶ This means the central repository does not need to hold sensitive administrative credentials for the end-user devices, significantly reducing the blast radius of a potential security breach.⁶

This model also changes the nature of IT work. Instead of spending time on repetitive manual tasks like resetting passwords or manually patching individual machines, IT professionals spend their time writing and refining

code.⁸ This shift towards a "Platform Engineering" mindset allows the IT team to build internal tools that empower other employees while maintaining strict security boundaries.¹⁵ The collaborative nature of pull requests means that junior engineers can receive constant feedback and mentorship from senior staff, improving the overall skill level of the department.⁸

The impact on disaster recovery cannot be overstated. In a manual world, a major system failure often requires a catastrophic rebuild effort, where administrators try to remember and recreate thousands of settings. With GitOps, the entire state of the fleet is saved in Git.⁹ Rebuilding the entire infrastructure is as simple as pointing the GitOps controller at a new set of resources and letting it "reconcile" the desired state.⁹ This capability provides IT leaders with a level of business continuity that was previously only available to the largest and most well-funded technology companies.

As organizations evaluate their tools, they should prioritize those that offer robust APIs and native integration with version control systems.² The goal is to create a seamless pipeline where a change in code is the only way to change the reality of the device fleet. This discipline reduces "cowboy engineering" and ensures that the organization's security posture is a reflection of its documented policies, not the ad-hoc actions of individual administrators.¹⁶

Finally, the shift to GitOps enables "Observability" in a way that traditional monitoring does not. While traditional monitoring tells you if a device is up, GitOps observability tells you if the device is correct.¹⁵ It provides a continuous feedback loop that confirms the observed state matches the desired state.¹⁵ This ensures that compliance is not a once-a-quarter audit event, but a continuous, real-time reality for the entire organization.⁴

What's next?

Learn how Fleet makes Infrastructure as Code for devices easy.

[Learn more](#)

Sources

1. Why Endpoint Management Has Become a Growing Challenge for Modern IT Teams, accessed February 28, 2026, <https://www.techtimes.com/articles/313483/20251218/why-endpoint-management-has-become-growing-challenge-modern-it-teams.htm>
2. 7 Common Endpoint Management Challenges - NinjaOne, accessed February 28, 2026, <https://www.ninjaone.com/blog/endpoint-management-challenges/>
3. 12 Common endpoint management challenges and their solutions - Scalefusion Blog, accessed February 28, 2026, <https://blog.scalefusion.com/endpoint-management-challenges/>
4. 10 Endpoint Management Challenges for CIOs and IT Teams - Trio MDM, accessed February 28, 2026, <https://www.trio.so/blog/endpoint-management-challenges>
5. What are Endpoint Security Management Challenges? - Palo Alto Networks, accessed February 28, 2026, <https://www.paloaltonetworks.com/cyberpedia/what-are-endpoint-security-management-challenges>
6. Top 10 benefits of implementing GitOps in your organisation - Adaptavist, accessed February 28, 2026, <https://www.adaptavist.com/blog/top-10-benefits-of-gitops>
7. What is Infrastructure as Code (IaC)? A Full Guide for Your Organization | Illumio, accessed February 28, 2026, <https://www.illumio.com/cybersecurity-101/infrastructure-as-code>
8. What I have learned from managing devices with GitOps - Fleet, accessed February 28, 2026, <https://fleetdm.com/guides/what-i-have-learned-from-managing-devices-with-gitops>
9. Enforce Secure Automated Deployment Practices through Infrastructure as Code, accessed February 28, 2026, <https://media.defense.gov/2024/Mar/07/2003407857/-1/-1/0/CSI-CLOUDTOP10-INFRASTRUCTURE-AS-CODE.PDF>
10. What Is Infrastructure as Code (IaC)? - Check Point Software, accessed February 28, 2026, <https://www.checkpoint.com/cyber-hub/cloud-security/what-is-infrastructure-as-code-iac/>
11. GitOps, The Principles and Why You Should Adopt It - Enix.io, accessed February 28, 2026, <https://enix.io/en/blog/gitops/>
12. How to Implement Infrastructure as Code: A Comprehensive Guide - Harness, accessed February 28, 2026, <https://www.harness.io/harness-devops-academy/how-to-implement-infrastructure-as-code>
13. What is GitOps? A developer's guide - Gatling, accessed February 28, 2026, <https://gatling.io/blog/what-is-gitops>
14. What is GitOps? - IBM, accessed February 28, 2026, <https://www.ibm.com/think/topics/gitops>
15. What is GitOps? - Red Hat, accessed February 28, 2026, <https://www.redhat.com/en/topics/devops/what-is-gitops>
16. What is GitOps? - GitLab, accessed February 28, 2026, <https://about.gitlab.com/topics/gitops/>



17. What is GitOps and how does it benefit CI/CD? - Atlassian Community, accessed February 28, 2026, <https://community.atlassian.com/forums/App-Central-articles/What-is-GitOps-and-how-does-it-benefit-CI-CD/ba-p/2169665>
18. MDM vs. EMM vs. UEM: Key Differences for Device Management - Rippling, accessed February 28, 2026, <https://www.rippling.com/blog/mdm-vs-emm>
19. What is Modern Endpoint Management? - Stripe OLT, accessed February 28, 2026, <https://stripeolt.com/knowledge-hub/expert-intel/what-is-modern-unified-endpoint-management/>
20. The benefits of GitOps workflows - GitLab, accessed February 28, 2026, <https://about.gitlab.com/topics/gitops/gitops-best-practices/>
21. Legacy Software vs Modern Alternatives: In-Depth Analysis - Stromasys, accessed February 28, 2026, <https://www.stromasys.com/resources/legacy-software-vs-modern-alternatives/>
22. How I Visualize Cloud Infrastructure Diagrams: Tools, Techniques, and Practical Tips, accessed February 28, 2026, <https://medium.com/@lisaellingtonwrites/how-i-visualize-cloud-infrastructure-diagrams-tools-techniques-and-practical-tips-a38d80fcd3a5>
23. Top 10 Best Endpoint Management Software in 2026 - Cyber Press, accessed February 28, 2026, <https://cyberpress.org/best-endpoint-management-software/>
24. Guidance for Generating Infrastructure-as-Code Templates from Architecture Diagrams on AWS, accessed February 28, 2026, <https://aws.amazon.com/solutions/guidance/generating-infrastructure-as-code-templates-from-architecture-diagrams-on-aws/>

